Sistema de Pedidos da Lanchonete

Relatório técnico — Trabalho Prático Integrador
Programação de Software Básico em C
Autores: Giovanna Borneo e Felipe Bezerra
Versão: 11 de novembro de 2026

1. Introdução e objetivos

Este trabalho apresenta um sistema de terminal em C para cadastrar e acompanhar pedidos de uma lanchonete. O tema foi escolhido para representar as tarefas propostas no enunciado: cada pedido tem uma identificação, uma descrição, um tempo de preparo e um estado.

O objetivo é reunir os conteúdos da disciplina em um programa simples: estruturas, vetores, ponteiros, arquivos, processos, comunicação entre processos e threads. O usuário pode cadastrar, listar, consultar e preparar pedidos, além de salvar os dados e consultar o histórico.

O preparo é uma simulação com `sleep()` de um ou dois segundos. Não existe integração com equipamentos de cozinha. Essa espera permite observar o início e o término da tarefa e comparar como o código organiza a execução normal, o processo filho e as threads.

2. Fundamentação teórica e requisitos

Usamos uma `struct` para reunir os campos de cada pedido e um vetor para guardar a lista. Os ponteiros permitem que as funções alterem um pedido ou o contador de registros do chamador.

Um processo possui seu próprio espaço de memória. Por isso, o filho criado com `fork()` precisa comunicar seu resultado ao pai. Já as threads pertencem ao mesmo processo e compartilham sua memória. O mutex controla o acesso à parte que não deve ser usada pelas duas threads ao mesmo tempo.

As funções do programa foram organizadas de acordo com os requisitos:

- **RF01 a RF04:** cadastrar, listar, consultar e executar o preparo.
- **RF05 e RF06:** salvar e carregar a lista em arquivo.
- **RF07 a RF09:** criar um processo, usar IPC por pipe e executar dois pedidos com threads.
- **RF10 e RF11:** registrar o histórico e medir uma operação repetida.
- **RF12:** tratar entradas inválidas e falhas dos recursos utilizados.

3. Arquitetura e desenvolvimento

Dividimos o código em módulos pequenos para separar as responsabilidades e facilitar a leitura:

Arquivo em src/	Responsabilidade
main.c	Menu, leitura das entradas, chamadas dos módulos e medição de consulta.
pedido.c	Cadastro, listagem, busca por ID e preparo do pedido.
arquivo.c	Salvar e carregar os pedidos em TXT.
processo.c	Criar o filho, comunicar pelo pipe e aguardar o término.
thread.c	Criar e aguardar as duas threads de preparo.
log.c	Registrar e mostrar o histórico, com mutex na gravação.

Os cabeçalhos ficam em `include/`, a documentação em `docs/` e os arquivos gerados em `dados/`. O Makefile compila os módulos com `-Wall -Wextra -pthread`. A execução deve partir da raiz do projeto: `make` e depois `./programa`.

A lista principal é um vetor de até 50 pedidos, mantido em `main()`. Cada registro possui ID único, descrição de até 99 bytes, tempo de preparo e estado PENDENTE ou PRONTO. A busca é linear: percorre as posições ocupadas até encontrar o ID. Escolhemos o vetor fixo por ser suficiente para o escopo e dispensar alocação dinâmica.

A função `prepararPedido()` é o núcleo da execução. Ela valida o pedido, aguarda o tempo indicado, muda o estado para PRONTO e tenta registrar o histórico. Essa mesma função é usada pelos três modos, evitando repetir a regra de preparo.

4. Processos, threads e IPC

Na opção 5, o programa cria um pipe e depois chama `fork()`. O filho prepara sua cópia do pedido e envia um inteiro com o estado final. O pai recebe a mensagem, aguarda o filho com `waitpid()` e só confirma PRONTO após verificar o resultado e o término normal. Cada processo fecha a ponta do pipe que não utiliza.

Na opção 6, duas threads recebem os endereços de pedidos diferentes. As duas são criadas antes dos `pthread_join()`, permitindo a sobreposição dos preparos. Como alteram elementos distintos e o menu aguarda o término, não usamos mutex nos campos dos pedidos nesse fluxo.

O histórico é compartilhado pelas threads. Por isso, o mutex protege a abertura, a escrita e o fechamento do arquivo, mas não a espera do preparo. O menu aguarda tanto o processo quanto as threads; não há atendimento de novos comandos em segundo plano.

5. Persistência, logs e testes

Persistência e histórico

Escolhemos TXT porque os registros são pequenos, o conteúdo pode ser conferido no editor e não é necessária uma biblioteca externa. O enunciado permite texto ou binário. Cada linha de `dados/pedidos.txt` segue o formato `ID;descricao;tempoPreparo;status`. Exemplo ilustrativo:

```
1;X-burger e suco;2;0
2;Batata frita;1;1
```

O estado 0 significa PENDENTE e 1, PRONTO. Como o ponto e vírgula separa os campos, ele não é aceito na descrição. Salvar e carregar são ações manuais: a opção 9 substitui o TXT, e a opção 10 substitui a lista em memória.

O carregamento usa um vetor temporário e só altera a lista atual depois das verificações. A leitura rejeita NUL embutido antes de interpretar a linha. Também são verificados formato, limite de registros e IDs repetidos.

O arquivo `dados/historico.txt` recebe data e hora, PID, ID do pedido, estado e duração. Ele acrescenta registros sem apagar os anteriores. O histórico registra eventos de preparo; não substitui o salvamento da lista.

Roteiro de testes

O enunciado exige pelo menos dez casos de teste, sem exigir automação. O roteiro pode ser executado pelo menu, com registro dos resultados observados e capturas de tela. A tabela apresenta os resultados esperados:

Caso	Resultado esperado
1. Cadastro válido	Pedido inserido na lista.
2. ID duplicado	Cadastro recusado sem alterar o original.
3. Consulta válida	Dados do pedido exibidos.
4. Consulta inexistente	Mensagem de não encontrado.
5 e 6. Salvar e carregar	Arquivo gravado e lista recuperada.
7 e 8. Processo e IPC	Filho criado e resultado recebido pelo pai.
9. Threads	Dois pedidos preparados.
10. Entrada inválida	Erro informado sem executar a operação inválida.

6. Desempenho, problemas e conclusão

Medição de desempenho

A opção 8 mede cinco consultas pelo mesmo ID. Para cada repetição, usamos `clock_gettime(CLOCK_MONOTONIC)` antes e depois da busca e calculamos a diferença em milissegundos. A leitura do usuário, a impressão e as estatísticas ficam fora do intervalo medido.

Depois, o programa apresenta os cinco tempos, o menor, o maior e a média. Cinco repetições atendem ao mínimo do enunciado, mas uma busca em até 50 registros é muito curta. A resolução do relógio, o cache e a carga do sistema influenciam os valores. Portanto, essa amostra não permite afirmar estabilidade nem ganho de velocidade com threads.

Conclusão

O projeto reúne os conteúdos de C e sistemas operacionais em um exemplo de fácil acompanhamento. A principal diferença demonstrada é que o filho precisa comunicar seu resultado, enquanto as threads compartilham o vetor e exigem cuidado com o histórico comum.

Referências

Enunciado do Trabalho Prático Integrador — Programação de Software Básico em C. Material fornecido pelo professor, especialmente requisitos, persistência, medição, testes e relatório (pp. 7, 12, 14, 18 e 20).